



D12 — Manual de Operacoes Exercito de Elite de Agentes

Integracao completa: Antigravity + Supabase + Vercel + GitHub

Times especializados, Skills, Comandos e Workflow Fluido

Projeto Aura — SaaS de Gestao de Eventos

Documento 12 — Specdrive Tecnico Final

Maio 2026



1. Visao Geral: A Maquina de Producao Aura

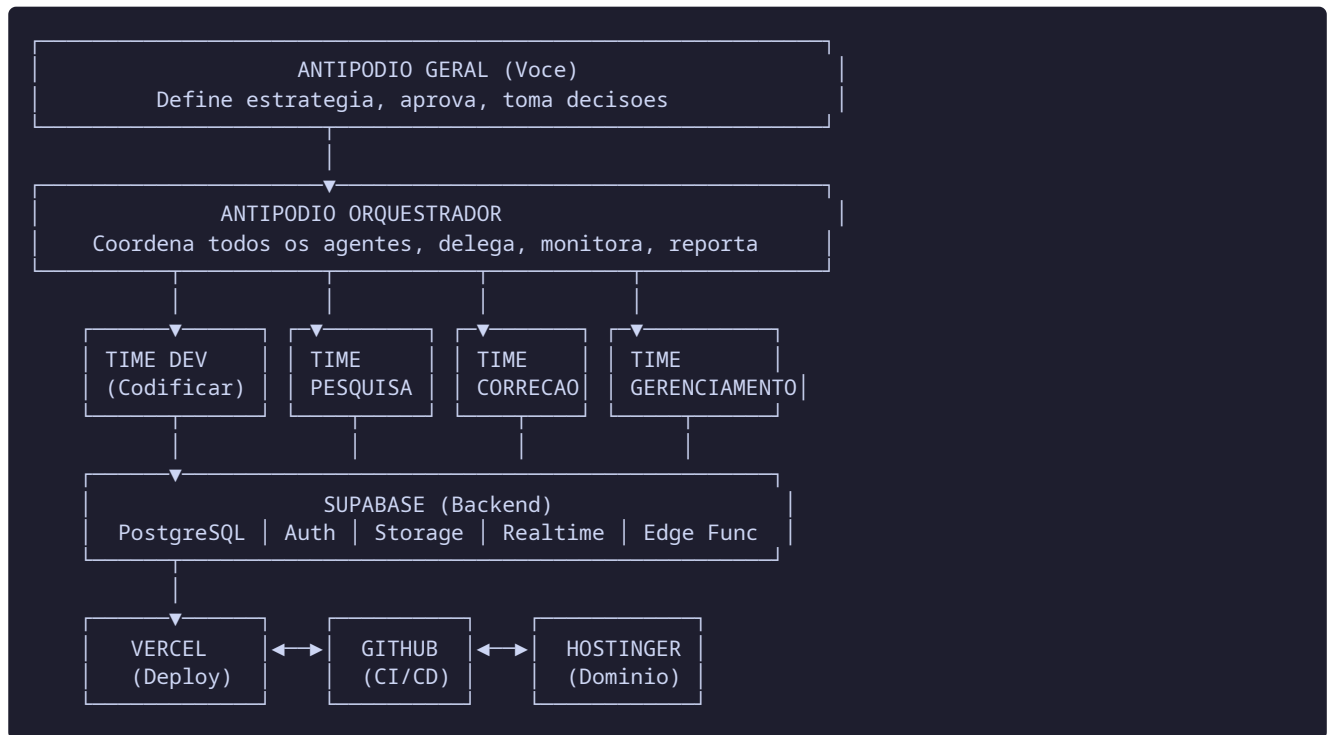
Este documento transforma o Aura de um projeto em uma **maquina de producao autonoma**. Aqui definimos o exercito completo de agentes de IA, cada um com uma funcao especifica, comandos proprios, skills especializadas e regras de colaboracao. O objetivo e eliminar conflitos, gargalos e retrabalho.

1.1 A Quatro Tecnologias do Ecosystema

Tabela 1 Papel de Cada Tecnologia no Ecosystema Aura

Tecnologia	Papel	Responsabilidade	Custo Est.
Antigravity	Quartel-general dos agentes	Desenvolvimento, orquestracao, skills, planejamento	Gratis
Supabase	Backend completo	Banco PostgreSQL, Auth, Storage, Realtime, Edge Functions	\$25/mes
Vercel	Infraestrutura de deploy	Hospedagem frontend, edge network, previews, analytics	\$20/mes
GitHub	Controle e CI/CD	Versionamento, Actions, code review, secrets management	Gratis (publico)

1.2 Como as 4 Tecnologias se Conectam



2. O Exercito de Elite: 6 Times, 12 Agentes

2.1 Estrutura Hierarquica

O exercito e organizado em **6 times especializados**, cada um com agentes dedicados. Nenhum agente faz tudo — cada um tem uma funcao especifica e um escopo delimitado.

Tabela 2 Estrutura do Exército de Agentes

Time	Agente	Funcao Principal	Cor
ORQUESTRACAO	Maestro	Coordena todos os times, define prioridades, resolve conflitos	●
	Radar	Monitora progresso, identifica bloqueios, gera relatorios	●
DESENVOLVIMENTO	Arquiteto	Design de schema, APIs, arquitetura de componentes	●
	Frontend	Implementa componentes React, telas, hooks, stores	●
	Backend	SQL, RLS policies, Edge Functions, triggers	●
PESQUISA	Investigador	Pesquisa melhores praticas, bibliotecas, patterns	●
	Analista	Analisa codigo, sugere otimizacoes, benchmarking	●
CORRECAO	Auditor	Audita seguranca (RLS, XSS, CSRF), revisa PRs	●
	Debugger	Corrige bugs, testa, valida funcionamento	●
GERENCIAMENTO	DevOps	Deploy, CI/CD, infraestrutura, variaveis de ambiente	●
	Documentador	Mantem documentacao atualizada, gera changelogs	●
LEITURA DE CODIGO	Cartografo	Mapeia codebase, entrega contexto, encontra dependencias	●

2.2 Time ORQUESTRACAO

Time ORQUESTRACAO ●

Os chefes. Tomam decisoes, coordenam acoes, resolvem conflitos.

AGENTE: Maestro CMD

Funcao: Coordenador supremo. Recebe missoes do usuario (voce), divide em tarefas, delega aos times e garante que tudo flua sem conflito.

Escopo: Nao escreve codigo. Apenas coordena. E o "scrum master" dos agentes.

Comandos:

- `MAESTRO INICIAR "[missao]"` — Inicia uma nova missao com descricao clara
- `MAESTRO STATUS` — Mostra o estado de todas as tarefas ativas
- `MAESTRO PRIORIZAR [tarefa]` — Move uma tarefa para o topo da fila
- `MAESTRO BLOQUEIO` — Lista todos os bloqueios ativos
- `MAESTRO RELATORIO` — Gera relatorio de progresso do dia
- `MAESTRO SYNC` — Sincroniza contexto entre todos os agentes

AGENTE: Radar CMD

Funcao: Monitor. Acompanha o progresso de cada time, detecta atrasos, identifica quando um agente esta "perdido" e precisa de ajuda.

Escopo: Nao executa tarefas. Apenas observa e reporta.

Comandos:

- `RADAR SCAN` — Verifica progresso de todas as tarefas
- `RADAR ALERTAS` — Lista problemas detectados
- `RADAR MONITOR [agente]` — Monitora um agente especifico
- `RADAR METRICAS` — Mostra velocidade, taxa de erro, cobertura

2.3 Time DESENVOLVIMENTO

Time DESENVOLVIMENTO ●

Quem constrói. Escrevem todo o código do projeto.

AGENTE: Arquiteto DEV

Funcao: Design de alto nivel. Define schemas de banco, estrutura de APIs, componentes e patterns antes do codigo ser escrito.

Skills: System design, PostgreSQL, React patterns, TypeScript

Comandos:

- `ARQUITETO SCHEMA [entidade]` — Design schema de uma entidade (ex: events, orders)
- `ARQUITETO API [endpoint]` — Design contrato de API (input/output)
- `ARQUITETO COMPONENTE [nome]` — Design estrutura de componente (props, state, hooks)
- `ARQUITETO REVISAR [arquivo]` — Revisa design de algo ja implementado

AGENTE: Frontend DEV

Funcao: Implementa componentes React, paginas, hooks, stores Zustand e integracoes TanStack Query.

Skills: React 19, TypeScript, Tailwind, shadcn/ui, Zustand, TanStack Query

Comandos:

- `FRONTEND CRIAR [componente] em [caminho]` — Cria novo componente
- `FRONTEND INTEGRAR [pagina]` — Substitui mock por API real
- `FRONTEND REFACTOR [arquivo]` — Refatora codigo existente
- `FRONTEND TESTAR [componente]` — Cria e roda testes
- `FRONTEND BUILD` — Executa build e corrige erros

AGENTE: Backend DEV

Funcao: Implementa SQL, RLS policies, triggers, Edge Functions e migrations.

Skills: PostgreSQL, Supabase, Deno/TypeScript (Edge Functions), SQL

Comandos:

- `BACKEND TABELA [nome] [campos]` — Cria tabela com campos
- `BACKEND RLS [tabela]` — Cria RLS policies para tabela
- `BACKEND TRIGGER [nome]` — Cria trigger
- `BACKEND EDGE [nome]` — Cria Edge Function
- `BACKEND MIGRATE` — Aplica migrations

2.4 Time PESQUISA

Time PESQUISA ●

Descobrem o melhor caminho antes de construir.

AGENTE: Investigador CMD

Funcao: Pesquisa o estado da arte. Encontra as melhores bibliotecas, patterns e solucoes antes do time DEV gastar tempo.

Comandos:

- `INVESTIGADOR BIBLIOTECA [criterios]` — Encontra melhor lib para o caso
- `INVESTIGADOR PATTERN [problema]` — Encontra pattern arquitetural ideal
- `INVESTIGADOR GATEWAY [requisitos]` — Pesquisa gateways de pagamento
- `INVESTIGADOR COMPARAR [opcaoA] vs [opcaoB]` — Compara solucoes

AGENTE: Analista CMD

Funcao: Analisa código já escrito para encontrar oportunidades de melhoria, dívida técnica e otimizações.

Comandos:

- `ANALISTA REVIEW [arquivo]` — Analisa qualidade do código
- `ANALISTA PERF [arquivo]` — Analisa performance
- `ANALISTA DEPT` — Lista tech debt do projeto

2.5 Time CORRECAO

Time CORRECAO ●

Guardiões da qualidade. Encontram e corrigem problemas.

AGENTE: Auditor SEC

Funcao: Audita segurança. Verifica RLS, XSS, CSRF, injeção SQL, vazamento de dados. É o guardião da segurança.

Skills: OWASP, PostgreSQL RLS, web security, Supabase security

Comandos:

- `AUDITOR RLS` — Audita todas as RLS policies
- `AUDITOR SEC` — Audita segurança geral do projeto
- `AUDITOR API [endpoint]` — Testa endpoint para vulnerabilidades
- `AUDITOR REVIEW [PR]` — Revisa pull request focado em segurança

AGENTE: Debugger SEC

Funcao: Encontra e corrige bugs. Executa testes, isola problemas, valida correções.

Comandos:

- `DEBUGGER ENCONTRAR [descricao do bug]` — Investiga e encontra causa
- `DEBUGGER CORRIGIR [bug]` — Implementa correção
- `DEBUGGER TESTAR [escopo]` — Executa testes completos
- `DEBUGGER REGRESSAO` — Verifica se correção não quebrou nada

2.6 Time GERENCIAMENTO

Time GERENCIAMENTO ●

Fazem a máquina funcionar: deploy, documentação, infraestrutura.

AGENTE: DevOps OPS

Função: Gerencia deploy, CI/CD, variáveis de ambiente, monitoramento e infraestrutura.

Skills: Vercel, GitHub Actions, Supabase CLI, DNS

Comandos:

- `DEVOPS DEPLOY [ambiente]` — Faz deploy (staging/produção)
- `DEVOPS CI` — Configura/atualiza GitHub Actions
- `DEVOPS ENV` — Gerencia variáveis de ambiente
- `DEVOPS MIGRATE [ambiente]` — Aplica migrations no ambiente

AGENTE: Documentador OPS

Função: Mantém toda a documentação sincronizada com o código.

Comandos:

- `DOC ATUALIZAR [doc]` — Atualiza documento após mudança
- `DOC GERAR [tipo]` — Gera documentação (API, componente)
- `DOC CHANGELOG` — Gera changelog desde última versão

2.7 Time LEITURA DE CÓDIGO

Time LEITURA DE CÓDIGO ●

O contexto. Entendem o código existente para entregar informação precisa.

AGENTE: Cartografo CMD

Funcao: Mapeia o codebase inteiro. Sabe onde cada arquivo esta, quem depende de quem, e entrega contexto completo quando outro agente precisa.

Comandos:

- CARTOGRAFO MAPEAR — Mapeia projeto inteiro (atualiza indice)
- CARTOGRAFO DEP [arquivo] — Encontra dependencias de um arquivo
- CARTOGRAFO USO [componente] — Onde este componente e usado
- CARTOGRAFO CONTEXTO [tarefa] — Entrega contexto completo para uma tarefa
- CARTOGRAFO CONFLITO [arquivoA] [arquivoB] — Detecta conflitos potenciais

3. Skills: O Conhecimento Especializado de Cada Agente

No Antigravity (e Kimi), uma **Skill** e um pacote de conhecimento especializado que um agente carrega quando precisa. Funciona como um "modo especialista" — o agente so carrega o conhecimento quando a tarefa exige.

3.1 Estrutura de uma Skill

No Antigravity, as skills ficam em:

```
# Skills do projeto (escopo local) .agent/skills/ aura-backend/SKILL.md aura-frontend/SKILL.md
aura-security/SKILL.md aura-devops/SKILL.md # Skills globais (disponiveis em todos os projetos)
~/.gemini/antigravity/skills/ react-patterns/SKILL.md supabase-rls/SKILL.md
```

3.2 Skill: aura-backend

```
.agent/skills/aura-backend/SKILL.md
```

3. Skills: O Conhecimento Especializado de Cada Agente

```
---
name: aura-backend
description: Use esta skill para todas as tarefas de backend do Aura – Supabase, PostgreSQL, RLS, Edge
---

# Aura Backend Skill

## Stack
- Supabase (PostgreSQL 15+)
- Edge Functions (Deno + TypeScript)
- Migrations SQL versionadas

## Schema do Banco
0 Aura tem 30 tabelas principais. Sempre consulte o schema completo
antes de criar ou modificar tabelas.

### Tabelas Core (criar primeiro)
1. profiles – usuarios (id, email, full_name, role, status)
2. producer_profiles – dados de produtores (user_id, company_name, cnpj)
3. events – eventos (producer_id, title, slug, category, status)
4. ticket_types – tipos de ingresso (event_id, name, price, qty)
5. ticket_orders – pedidos (user_id, event_id, total_amount, status)
6. ticket_order_items – itens de pedido (order_id, ticket_type_id, qty)

### Regras de Schema
- Toda tabela TEM que ter: id (uuid), created_at, updated_at
- Chaves estrangeiras sempre com ON DELETE CASCADE ou RESTRICT explicito
- Nunca use nomes de tabelas no plural misturado (escolha: snake_case)
- Enums devem ser criados como TYPE antes das tabelas

## RLS Patterns

### Pattern Basico (usuario ve so seus dados)
CREATE POLICY "Users read own" ON table_name
  FOR SELECT TO authenticated
  USING ((select auth.uid()) = user_id);

### Pattern de Produtor (produtor ve so seus eventos)
CREATE POLICY "Producers read own events" ON events
  FOR SELECT TO authenticated
  USING ((select auth.uid()) = producer_id);

### Pattern Admin (admin ve tudo)
CREATE POLICY "Admins read all" ON table_name
  FOR SELECT TO authenticated
  USING (
    EXISTS (
      SELECT 1 FROM profiles
      WHERE id = (select auth.uid()) AND role = 'admin'
    )
  );

### Regras Inquebraveis
- SEMPRE use (select auth.uid()) em vez de auth.uid() (performance)
- SEMPRE especifique TO authenticated ou TO anon
- NUNCA use USING (true) – expoe todos os dados
- Toda tabela DEVE ter RLS habilitado
- SEMPRE crie policieis para SELECT, INSERT, UPDATE ( DELETE so se necessario)

## Edge Function Template

```typescript
// Template padrao para Edge Functions
import { createClient } from "@supabase/supabase-js";
```

### 3.3 Skill: aura-frontend

```
.agent/skills/aura-frontend/SKILL.md
```

### 3. Skills: O Conhecimento Especializado de Cada Agente

```

name: aura-frontend
description: Use esta skill para todas as tarefas de frontend do Aura – componentes React, hooks, stor

Aura Frontend Skill

Stack
- React 19 + TypeScript + Vite
- Tailwind CSS + shadcn/ui
- Zustand (estado client)
- TanStack Query (estado server)
- React Hook Form + Zod (formularios)

Estrutura de Componentes

Novo Componente – Template
```tsx
// src/components/NomeComponente.tsx
import { useState } from "react";
import { Button } from "@/components/ui/button";

interface NomeComponenteProps {
  title: string;
  onAction: () => void;
  isLoading?: boolean;
}

export function NomeComponente({ title, onAction, isLoading }: NomeComponenteProps) {
  const [internalState, setInternalState] = useState("");

  if (isLoading) return <NomeComponenteSkeleton />;

  return (
    <div className="rounded-lg border p-4">
      <h3 className="text-lg font-semibold">{title}</h3>
      <Button onClick={onAction} disabled={isLoading}>
        Acao
      </Button>
    </div>
  );
}

function NomeComponenteSkeleton() {
  return <div className="animate-pulse rounded-lg bg-gray-200 h-20" />;
}
```

Novo Hook TanStack Query – Template
```tsx
// src/hooks/useEntidade.ts
import { useQuery, useMutation, useQueryClient } from "@tanstack/react-query";
import { supabase } from "@/lib/supabase";

const ENTIDADE_KEY = "entidade";

export function useEntidades() {
  return useQuery({
    queryKey: [ENTIDADE_KEY],
    queryFn: async () => {
      const { data, error } = await supabase
        .from("tabela")
        .select("*")
        .order("created_at", { ascending: false });
    }
  });
}
```

3.4 Skill: aura-security

```
.agent/skills/aura-security/SKILL.md
```

```
---
name: aura-security
description: Use esta skill para auditar seguranca do Aura – RLS policies, Edge Functions, inputs, au
---

# Aura Security Skill

## Checklist de Seguranca (rodar antes de cada deploy)
- [ ] Todas as tabelas tem RLS habilitado
- [ ] Nenhuma policy usa USING (true)
- [ ] Todas as policies usam (select auth.uid())
- [ ] Edge Functions validam JWT antes de processar
- [ ] Inputs de usuario sao sanitizados (DOMPurify)
- [ ] Nenhuma chave service_role no frontend
- [ ] Variaveis de ambiente usam prefixo VITE_
- [ ] Webhooks de pagamento validam assinatura
- [ ] Uploads limitados a 5MB e verificam tipo MIME

## OWASP Top 10 – Mitigacoes
1. Injection – Use queries parametrizadas (Supabase client ja faz)
2. Broken Auth – Use Supabase Auth, nao reinvente
3. Sensitive Data – Nunca logue dados sensiveis
4. XXE – Nao processe XML do usuario
5. Broken Access – RLS em TODAS as tabelas
6. Security Misconfig – Headers de seguranca no Vercel
7. XSS – Escape todos os dados do usuario (React ja faz)
8. Insecure Deserialization – Nao use eval() ou JSON.parse sem validar
9. Known Vulnerabilities – Rode npm audit regularmente
10. Insufficient Logging – Tabela audit_logs para todas as operacoes financeiras
```

3.5 Skill: aura-devops

```
.agent/skills/aura-devops/SKILL.md
```

```

---
name: aura-devops
description: Use esta skill para tarefas de deploy, CI/CD, infraestrutura e operacoes do Aura. Inclui
---

# Aura DevOps Skill

## Ambientes
- **Local:** http://localhost:5173 (Vite dev server)
- **Staging:** https://staging.aura.tickets (Vercel preview)
- **Producao:** https://aura.tickets (Vercel production)

## Deploy Vercel
```bash
Deploy manual
vercel --prod

Deploy staging
vercel
```

## Variaveis de Ambiente (Vercel)
VITE_SUPABASE_URL – URL do projeto Supabase
VITE_SUPABASE_ANON_KEY – Chave anonima (publica, segura)
VITE_APP_URL – URL da aplicacao

## GitHub Actions Workflow
- CI roda em todo PR: lint, typecheck, test, build
- CD roda em push para main: deploy producao
- CD roda em push para develop: deploy staging

## Supabase CLI
```bash
Login
supabase login

Link projeto
supabase link --project-ref <ref>

Aplicar migrations
supabase db push

Deploy Edge Functions
supabase functions deploy

Status
supabase migration list
```

```

4. Workflow Sem Conflitos: O Ciclo de Vida de uma Tarefa

Este é o coração do sistema. Um workflow que garante que **nenhum agente pise no outro** e que **cada tarefa flua sem bloqueios**.

4.1 O Ciclo de Vida

```
FASE 0: RECEPCAO (Maestro recebe missao do usuario)
|
FASE 1: MAPEAMENTO (Cartografo entrega contexto completo)
|
FASE 2: PESQUISA (Investigador + Analista encontram melhor caminho)
|
FASE 3: DESIGN (Arquiteto define schema, API, componente)
|
FASE 4: IMPLEMENTACAO (Frontend + Backend escrevem codigo)
|
FASE 5: AUDITORIA (Auditor revisa seguranca e qualidade)
|
FASE 6: CORRECAO (Debugger corrige bugs encontrados)
|
FASE 7: INTEGRACAO (Frontend integra com Backend)
|
FASE 8: DEPLOY (DevOps faz deploy em staging)
|
FASE 9: VALIDACAO (Debugger testa em staging)
|
FASE 10: ENTREGA (DevOps faz deploy em producao)
|
FASE 11: DOCUMENTACAO (Documentador atualiza docs)
```

4.2 Regras Anti-Conflito

Tabela 3 Regras para Evitar Conflitos entre Agentes

| Regra | Descricao | Quem aplica |
|--|---|-------------|
| R1: Nenhum agente escreve sem design previo | Arquiteto DEVE aprovar o design antes de Frontend/Backend codificar | Maestro |
| R2: Um arquivo, um agente por vez | Dois agentes nunca editam o mesmo arquivo simultaneamente | Cartografo |
| R3: Backend antes de Frontend | Backend cria API e schema antes de Frontend consumir | Maestro |
| R4: Auditoria antes de deploy | Auditor revisa TODO codigo antes de ir para staging | Maestro |
| R5: Testar antes de entregar | Debugger valida em staging antes de deploy em producao | Maestro |
| R6: Documentar apos entregar | Documentador atualiza docs apenas apos producao validada | Maestro |
| R7: Contexto sempre atualizado | Cartografo atualiza o mapa do projeto a cada 5 tarefas | Radar |
| R8: Bloqueio = escalar | Se um agente ficar bloqueado por mais de 10 min, Maestro escala | Radar |

4.3 Exemplo Real: Tarefa "Criar Sistema de Check-in"

FASE 0 - RECEPCAO:

Maestro recebe: "Preciso de um sistema de check-in com QR code para os eventos"

FASE 1 - MAPEAMENTO:

Cartografo: "Sistema atual tem: pagina de check-in (visual), componente QR scanner, tabela ticket_order_items com campo checkin_at. Falta: API de validacao, integracao camera, update em tempo real."

FASE 2 - PESQUISA:

Investigador: "Melhor lib para QR: html5-qrcode (42k estrelas). Melhor pattern: validar no Edge Function, atualizar via Realtime."

Analista: "Tabela ticket_order_items ja tem checkin_at e checkin_by. So precisa da logica de validacao."

FASE 3 - DESIGN:

Arquiteto: "Edge Function checkin-scan: recebe QR code, valida no banco, retorna status.
Frontend: usa html5-qrcode para ler, chama Edge Function, mostra resultado."

FASE 4 - IMPLEMENTACAO:

Backend: Cria Edge Function checkin-scan com validacao JWT e update no banco.
Frontend: Integra html5-qrcode, cria hook useCheckin, atualiza pagina de check-in.

FASE 5 - AUDITORIA:

Auditor: "RLS ok? Sim. JWT validado? Sim. Input sanitizado? Sim. Aprovado."

FASE 6 - CORRECAO:

Debugger: "Bug encontrado: QR code expirado nao retorna mensagem clara. Corrigido."

FASE 7 - INTEGRACAO:

Frontend: Integra com Backend. Teste end-to-end: ler QR → chamar API → mostrar resultado.

FASE 8-10 - DEPLOY:

DevOps: Deploy staging → teste → deploy producao.

FASE 11 - DOCUMENTACAO:

Documentador: Atualiza D8 com o novo hook useCheckin e Edge Function.

5. Configurando o Antigravity para o Aura

5.1 Instalacao do Antigravity

Passo 1: Acesse `antigravity.dev` e faça login com sua conta Google.

Passo 2: Na primeira abertura, selecione:

- **Development mode:** "Agent-assisted development" (recomendado — voce controla, a IA ajuda)
- **Terminal Policy:** "Auto" (agente executa comandos padrao sem perguntar)
- **Modelo:** Gemini 3 Pro (otimizado para codigo e planejamento multi-etapas)

Passo 3: Importe o projeto Aura (do GitHub ou upload de pasta).

5.2 Instalando o Antigravity Kit 2.0

O **Kit 2.0** adiciona agentes especializados e skills automaticas. Para instalar:

Passo 4: No Agent Manager, clique em "Install Kit" ou "Add Skills".

Passo 5: Selecione "Antigravity Kit 2.0" e confirme. O kit adiciona:

- Agente Backend (SQL, API, database)
- Agente Frontend (UI, React, CSS)
- Agente DevOps (deploy, CI/CD)
- Skills pre-configuradas para cada agente
- Workflows de planejamento automatico

Passo 6: Responda as perguntas de configuracao:

- **Estilo de design:** Escolha "Modern Dark/Light" (o Aura ja usa esse tema)
- **Sistema de autenticacao:** Escolha "Supabase Auth" (integra com RLS)
- **Banco de dados:** Escolha "Supabase PostgreSQL"

5.3 Configurando as Skills do Aura

Passo 7: Crie a pasta de skills na raiz do projeto:

```
mkdir -p .agent/skills/aura-backend mkdir -p .agent/skills/aura-frontend mkdir -p  
.agent/skills/aura-security mkdir -p .agent/skills/aura-devops
```

Passo 8: Copie o conteudo das skills das Secoes 3.2 a 3.5 para cada arquivo SKILL.md correspondente.

Passo 9: Faça upload dos documentos D1 a D11 como contexto no Antigravity. Na secao "Knowledge" ou "Context", adicione os PDFs.

6. Supabase: O Coracao do Backend

6.1 O que o Supabase Faz no Aura

Tabela 4 Componentes do Supabase e seu Papel no Aura

| Componente | Funcao no Aura | Status |
|----------------|---|------------------------|
| PostgreSQL | Banco de dados relacional com 30 tabelas | A criar (D5) |
| Auth | Login/registro com email, Google, Facebook | A configurar |
| Storage | Upload de avatares, banners, certificados | A configurar |
| Realtime | Atualizacoes em tempo real (check-in, status pagamento) | A habilitar |
| Edge Functions | Logica serverless (pagamentos, webhooks, relatorios) | A criar (10 funcoes) |
| RLS | Seguranca — cada usuario ve so seus dados | A criar (50+ policies) |

6.2 Melhores Praticas do Supabase (2025)

1. RLS em TODAS as tabelas — sem excecao

Toda tabela no schema public DEVE ter RLS habilitado. Sem RLS, qualquer pessoa com a anon key pode ler TUDO. Verifique: `SELECT tablename FROM pg_tables WHERE schemaname = 'public' AND NOT rowsecurity;`

2. Use (select auth.uid()) para performance

Em RLS policies, use `(select auth.uid()) = user_id` em vez de `auth.uid() = user_id`. O select avalia uma vez por query; sem select, avalia para cada linha. Diferenca de 3 minutos para 2ms em tabelas grandes.

3. Security Definer Functions para permissoes complexas

Para checks de permissao que envolvem multiplas tabelas, crie functions com `SECURITY DEFINER`. Isso evita que o RLS avalie politicas em tabelas intermediarias, acelerando a query.

4. Views com security_invoker (PostgreSQL 15+)

Se criar views, sempre use `WITH (security_invoker = true)`. Sem isso, a view executa como o criador (postgres) e ignora RLS.

5. Migrations versionadas com CLI

Nunca faça alteracoes manuais no banco de producao. Use `supabase db push` com migrations versionadas em Git. Isso permite rollback e auditoração.

6. Realtime so onde necessario

Habilite Realtime apenas nas tabelas que realmente precisam. Cada tabela com Realtime consome recursos. Desabilite UPDATE e DELETE se so precisar de INSERT.

6.3 Configuracao do Supabase para o Aura

```
# 1. Criar projeto supabase projects create aura-platform --org-id SUA_ORG --region sa-east-1 --plan free # 2. Linkar projeto local supabase link --project-ref SEU_PROJECT_REF # 3. Habilitar extensoes necessarias supabase db execute "CREATE EXTENSION IF NOT EXISTS pg_trgm;" supabase db execute "CREATE EXTENSION IF NOT EXISTS pg_stat_statements;" # 4. Aplicar migrations supabase db push # 5. Deploy Edge Functions supabase functions deploy process-payment supabase functions deploy stripe-webhook supabase functions deploy send-email supabase functions deploy checkin-scan supabase functions deploy generate-report supabase functions deploy export-data # 6. Configurar Storage buckets supabase buckets create avatars --public supabase buckets create banners --public supabase buckets create certificates --public supabase buckets create exports --private # 7. Habilitar Realtime nas tabelas necessarias supabase realtime enable ticket_orders supabase realtime enable ticket_order_items supabase realtime enable notifications
```

7. Vercel: A Maquina de Deploy

7.1 Por que Vercel para o Aura

O Vercel e a plataforma ideal para o Aura porque:

- **Zero config:** Detecta React/Vite automaticamente e configura o build
- **Edge Network:** Deploy global — seu site carrega rapido em qualquer lugar do mundo
- **Previews automaticos:** Cada PR gera um link de preview unico
- **Analytics integrado:** Core Web Vitals medidos de dispositivos reais
- **Serverless Functions:** API routes se precisar (embora o Aura use Supabase Edge Functions)

7.2 Configuracao do Vercel para o Aura

```
vercel.json
```

```
{
  "version": 2,
  "name": "aura-platform",
  "framework": "vite",
  "buildCommand": "npm run build",
  "outputDirectory": "dist",
  "rewrites": [{ "source": "/(.*)", "destination": "/index.html" }],
  "headers": [
    {
      "source": "/assets/(.*)",
      "headers": [{ "key": "Cache-Control", "value": "public, max-age=31536000, immutable" }]
    },
    {
      "source": "/(.*)",
      "headers": [
        { "key": "X-Content-Type-Options", "value": "nosniff" },
        { "key": "X-Frame-Options", "value": "DENY" },
        { "key": "X-XSS-Protection", "value": "1; mode=block" },
        { "key": "Referrer-Policy", "value": "strict-origin-when-cross-origin" }
      ]
    }
  ],
  "env": {
    "VITE_SUPABASE_URL": "@supabase-url",
    "VITE_SUPABASE_ANON_KEY": "@supabase-anon-key"
  }
}
```

7.3 Variaveis de Ambiente no Vercel

Tabela 5 Variaveis de Ambiente do Aura no Vercel

| Variavel | Valor | Ambiente |
|------------------------|---------------------------------|------------|
| VITE_SUPABASE_URL | https://seu-projeto.supabase.co | All |
| VITE_SUPABASE_ANON_KEY | eyJhbGciOiJIUzI1NiIs... | All |
| VITE_APP_URL | https://aura.tickets | Production |
| VITE_APP_ENV | production / staging | All |

8. GitHub: O Sistema Nervoso

8.1 Estrategia de Branches

main ← Producao estavel (deploy automatico na Vercel) develop ← Desenvolvimento integrado (deploy staging) feature/* ← Cada nova funcionalidade hotfix/* ← Correcoes urgentes em producao

8.2 GitHub Actions Completo

```
.github/workflows/ci-cd.yml
```

```

name: Aura CI/CD Pipeline

on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main, develop]

concurrency:
  group: ${ github.workflow }-${ github.ref }
  cancel-in-progress: true

jobs:
  lint:
    name: Lint e Type Check
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v6
      - uses: actions/setup-node@v6
        with: { node-version: '20', cache: 'npm' }
      - run: npm ci
      - run: npm run lint
      - run: npx tsc --noEmit

  test:
    name: Testes
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v6
      - uses: actions/setup-node@v6
        with: { node-version: '20', cache: 'npm' }
      - run: npm ci
      - run: npm run test -- --coverage --watchAll=false
      - uses: codecov/codecov-action@v5
        with: { files: ./coverage/lcov.info, fail_ci_if_error: false }

  build:
    name: Build
    runs-on: ubuntu-latest
    needs: [lint, test]
    steps:
      - uses: actions/checkout@v6
      - uses: actions/setup-node@v6
        with: { node-version: '20', cache: 'npm' }
      - run: npm ci
      - run: npm run build
      - uses: actions/upload-artifact@v4
        with: { name: build-output, path: dist/, retention-days: 7 }

  deploy-supabase:
    name: Deploy Supabase
    runs-on: ubuntu-latest
    if: github.ref == 'refs/heads/main'
    steps:
      - uses: actions/checkout@v6
      - uses: supabase/setup-cli@v1
        with: { version: latest }
      - run: supabase link --project-ref ${ secrets.SUPABASE_PROJECT_REF }
        env: { SUPABASE_ACCESS_TOKEN: ${ secrets.SUPABASE_ACCESS_TOKEN } }
      - run: supabase db push
        env: { SUPABASE_ACCESS_TOKEN: ${ secrets.SUPABASE_ACCESS_TOKEN } }
      - run: supabase functions deploy
        env: { SUPABASE_ACCESS_TOKEN: ${ secrets.SUPABASE_ACCESS_TOKEN } }

```

8.3 GitHub Secrets Necessarios

Tabela 6 Secrets do GitHub para o Aura

| Secret | Valor | Onde pegar |
|-----------------------|-----------|---|
| SUPABASE_ACCESS_TOKEN | sbp_... | Supabase Dashboard → Account → Access Tokens |
| SUPABASE_PROJECT_REF | abc123def | URL do projeto: https://app.supabase.com/project/REF |
| VERCEL_TOKEN | ... | Vercel Dashboard → Settings → Tokens |
| VERCEL_ORG_ID | ... | vercel.json gerado localmente |
| VERCEL_PROJECT_ID | ... | vercel.json gerado localmente |

9. Prompts Prontos para o Exercito

9.1 Prompts para o Maestro (Orquestrador)

MISSAO 1 - Iniciar Projeto:

Voce e o Maestro do projeto Aura, uma plataforma de gestao de eventos (SaaS de ticketing). Sua missao e coordenar 12 agentes especializados em 6 times para construir o backend completo.

Etapas:

1. Chame CARTOGRAFO MAPEAR para entender o estado atual
2. Priorize as tarefas do D2 (Gap Analysis)
3. Delege ao time DEV: criar schema Supabase (D5)
4. Monitore progresso com RADAR SCAN a cada etapa
5. Antes de deploy, chame AUDITOR SEC

Nao escreva codigo. Apenas coordene e delegue.

MISSAO 2 - Nova Funcionalidade:

Nova missao: "Implementar sistema de cupons de desconto".

Execute o ciclo completo:

1. FASE 1: CARTOGRAFO CONTEXTO "cupons de desconto"
2. FASE 2: INVESTIGADOR PATTERN "discount coupon system"
3. FASE 3: ARQUITETO SCHEMA "coupons"
4. FASE 4: BACKEND TABELA coupons + FRONTEND CRIAR CouponInput
5. FASE 5: AUDITOR RLS
6. FASE 6: DEBUGGER TESTAR cupons
7. FASE 8-10: DEVOPS DEPLOY staging
8. FASE 11: DOC ATUALIZAR D5

9.2 Prompts para o Time DEV

BACKEND - Criar Schema:

AGENTE: Backend

MISSAO: Criar a tabela "events" no Supabase seguindo o D5.

Requisitos:

- Campos: id (uuid), producer_id (ref profiles), title, slug (unico), description, category (enum), banner_url, venue_name, venue_address, venue_city, venue_state, start_date, end_date, status (enum: draft/published/cancelled/finished), settings (jsonb), created_at, updated_at
- RLS: produtor ve seus eventos, admin ve tudo, anonimo ve so published
- Trigger: updated_at atualiza automaticamente
- Indice: slug (unico), producer_id, status, start_date

Use a skill aura-backend. Crie migration em supabase/migrations/. Execute supabase db push e corrija erros.

FRONTEND - Integrar Pagina:

AGENTE: Frontend

MISSAO: Substituir mocks da pagina EventList por dados reais do Supabase.

Requisitos:

1. Criar hook useEvents em src/hooks/useEvents.ts (TanStack Query)
2. Substituir mockData.events na pagina EventList
3. Implementar loading skeleton
4. Implementar erro com retry
5. Paginacao: 12 eventos por pagina
6. Filtros: cidade, categoria, data, busca

Use a skill aura-frontend. Execute npm run build e corrija TODOS os erros.

9.3 Prompts para o Time PESQUISA

INVESTIGADOR - Gateway de Pagamento:

AGENTE: Investigador

MISSAO: Pesquisar as melhores opcoes de gateway de pagamento para o Aura no Brasil.

Requisitos do projeto:

- Split de pagamentos (produtor recebe direto)
- Pix + Cartao de credito
- Taxas competitivas
- API REST bem documentada
- Suporte a webhooks

Entregavel: Relatorio comparativo com 3 opcoes, pros/contras, custos, e recomendacao.

9.4 Prompts para o Time CORRECAO

AUDITOR - Auditoria Completa:

AGENTE: Auditor

MISSAO: Auditoria de seguranca completa do projeto.

Verifique:

1. TODAS as tabelas tem RLS habilitado?
2. Nenhuma policy usa USING (true)?
3. Edge Functions validam JWT?
4. Variaveis de ambiente estao seguras?
5. Nenhuma chave service_role vazada no frontend?

Use a skill aura-security. Entregue relatorio com itens aprovados e itens a corrigir.

9.5 Prompts para o Time GERENCIAMENTO

DEVOPS - Pipeline Completo:

AGENTE: DevOps

MISSAO: Configurar CI/CD completo no GitHub Actions.

Requisitos:

1. CI: lint + typecheck + test em todo PR
2. CD staging: deploy em push para develop
3. CD producao: deploy em push para main (apenas se CI passar)
4. Deploy Supabase: migrations + Edge Functions
5. Secrets: configurar todos no GitHub

Use a skill aura-devops. Teste com um push para develop.

10. Checklist de Lançamento do Aura

Tabela 7 Checklist Completo para Colocar o Aura em Producao

| Fase | Tarefa | Agente | Status |
|------------------------|--|--------------------|--------|
| Banco de Dados | Criar projeto Supabase | DevOps | [] |
| | Executar migration init_schema.sql (30 tabelas) | Backend | [] |
| | Executar migration rls_policies.sql (50+ policies) | Backend | [] |
| | Executar migration triggers.sql (15 triggers) | Backend | [] |
| Auth | Configurar Supabase Auth (email + Google) | Backend | [] |
| | Implementar login/registro no frontend | Frontend | [] |
| | Testar fluxo completo de autenticacao | Debugger | [] |
| Eventos | Integrar listagem de eventos (API real) | Frontend | [] |
| | Integrar detalhe do evento | Frontend | [] |
| | Implementar filtros e busca | Frontend | [] |
| Ingressos | Integrar tipos de ingresso | Frontend | [] |
| | Implementar carrinho (Zustand) | Frontend | [] |
| | Criar componentes de pagamento (D10) | Frontend | [] |
| Pagamentos | Criar Edge Functions de pagamento | Backend | [] |
| | Implementar checkout com Pix | Frontend + Backend | [] |
| | Implementar checkout com Cartao | Frontend + Backend | [] |
| | Configurar webhooks | Backend | [] |
| Check-in | Implementar leitura de QR code | Frontend | [] |
| | Criar Edge Function de validacao | Backend | [] |
| Painel Produtor | Dashboard financeiro com graficos | Frontend | [] |
| | Relatorios de vendas e check-in | Frontend + Backend | [] |
| Seguranca | Auditar RLS (todas as tabelas) | Auditor | [] |
| | Auditar Edge Functions (JWT) | Auditor | [] |
| | Testar XSS e injecao | Auditor | [] |
| Deploy | Configurar Vercel (producao + staging) | DevOps | [] |
| | Configurar GitHub Actions (CI/CD) | DevOps | [] |

| | | | |
|--------------|----------------------------------|--------------|-----|
| Deploy | Configurar dominio (Hostinger) | DevOps | [] |
| Documentacao | Atualizar todos os docs (D1-D12) | Documentador | [] |

11. Resumo do Sistema Completo

O Aura agora tem um **exercito de 12 agentes** organizados em **6 times especializados**, com **4 skills customizadas**, integrando **4 tecnologias** (Antigravity, Supabase, Vercel, GitHub), seguindo um **workflow de 11 fases** com **8 regras anti-conflito**.

Tabela 8 Numeros do Sistema Aura

| Métrica | Valor |
|-----------------------------|--|
| Agentes | 12 (Maestro, Radar, Arquiteto, Frontend, Backend, Investigador, Analista, Auditor, Debugger, DevOps, Documentador, Cartografo) |
| Times | 6 (Orquestracao, Desenvolvimento, Pesquisa, Correcao, Gerenciamento, Leitura deCodigo) |
| Skills | 4 (aura-backend, aura-frontend, aura-security, aura-devops) |
| Tecnologias | 4 (Antigravity, Supabase, Vercel, GitHub) |
| Fases do workflow | 11 (da Recepcao a Documentacao) |
| Regras anti-conflito | 8 (de design previo a contexto atualizado) |
| Documentos do specdrive | 12 (D1 a D12) |
| Paginas totais do specdrive | 177+ paginas |

O Sistema Aura esta pronto para producao autonoma. Com este manual, voce pode:

1. Configurar o Antigravity com todas as skills
2. Delegar tarefas ao exercito de agentes
3. Acompanhar o progresso via Maestro e Radar
4. Garantir qualidade via Auditor e Debugger
5. Fazer deploy automatico via DevOps
6. Manter documentacao sincronizada via Documentador

Nenhum conflito. Nenhum gargalo. Apenas execucao.